

Project Title: Secure Pay: A GPT-4-Powered Banking Platform for Real-Time Fraud Detection and Financial Analytics Secure Pay is a secure, AI-enhanced online banking solution that utilizes GPT-4 for intelligent fraud detection. The system integrates user management, transaction monitoring, and financial analytics into a single, user-friendly interface accessible across multiple devices.

Table of Contents

1	Introduction.....	3
2	Background.....	3
3	Rationale.....	4
4	Ethical Considerations.....	5
5	Aim.....	5
6	Objectives.....	5
7	Risks.....	6
8	Literature Survey.....	7
8.1	AI in Financial Fraud Detection.....	7
8.2	Natural Language Processing (NLP) and Large Language Models (LLMs).....	8
8.3	Real-Time Transaction Monitoring and Analytics.....	8
8.4	Secure Authentication and Data Protection.....	8
8.5	Ethical AI Deployment and Bias Mitigation.....	9
8.6	ChatGPT-4 as an Intelligent Fraud Agent.....	9
8.7	Role of Admin Dashboards and User Interfaces.....	9
8.8	Comparative Review of Existing Fraud Detection Platforms.....	9
9	Methodology.....	11
9.1	Development Approach.....	11
9.2	GPT-4 API vs Custom Model: Rationale.....	12
9.3	System Architecture.....	13
9.4	Data Collection and Processing.....	13
9.5	Testing Methodology.....	14
9.6	Compatibility and Integration Challenges.....	15
9.7	Feedback Loop and Model Refinement.....	16
9.8	Future Improvements.....	16
10	Requirements.....	19
11	Design.....	20
12	Glossary.....	22
13	References.....	24

1 Introduction

SecurePay is a banking platform that uses AI to provide both secured and intelligent banking services to its users. In simple terms, SecurePay relies on the GPT-4 API to watch for any suspicious activity in transactions and catch any changes quickly. Since financial fraud is getting more complicated, SecurePay uses AI to keep up and deal with new kinds of fraud. You get both admin and user interfaces which allow ease in handling users, watching finances and lowering risks.

SecurePay is structured so that its frontend is done with React, the backend is built with Node.js and it stores data securely using MongoDB/PostgreSQL. Integrating ChatGPT-4 facilities into the platform, the system automatically detects suspicious activities and brings them to attention. There are user dashboards for more control, an admin management area, visual finance graphs and safe password authentication methods. In addition, people can look at their transactions in detail, handle their savings accounts and get alerts about suspected fraud right away.

The simulated data set is used to build and train the fraud detection model which is then tested with real transaction simulations. It also features secure RESTful APIs and safe encryption of information to comply with today's cybersecurity rules. SecurePay makes it safer for digital banking with advanced explainable AI and combination of fraud analysis.

2 Background

There are increasingly clever and harmful fraudsters attacking the financial sector. Because traditional rule-based systems cannot adapt quickly, they do not always avoid possible losses from new kinds of fraud. Because of these problems, AI and LLMs like ChatGPT-4 are being examined as smart, flexible solutions for protecting finances. Thanks to the GPT-4 API, SecurePay is now capable of continually watching for and detecting any problems in financial transactions as they happen.

Due to the rise of digital banking and more people using mobile and web financial tools, there are now more ways for hackers to attack. SmartBanking deals with this challenge by including a GPT-4 fraud detection system in all its standard features. Built differently from static models, the GPT-4 agent adjusts its knowledge as it learns from ongoing transactional events and finds new cases of fraud without simply following pre-set rules.

The system is designed to find fraud and also produces comprehensive reports, helping administrators quickly review a company's finances and risks. Using this method improves the function of bank platforms and also helps digital financial ecosystems become more trustworthy and clearer. SecurePay links what AI can do with what is needed to keep finances secure

3 Rationale

Since financial systems are becoming digital, cyber fraud is increasing quickly. Older fraud detection systems are not designed to catch highly developed and mutating fraud threats. An adaptive and intelligent approach is evidently necessary. SecurePay achieves this through using OpenAI's GPT-4 API to act as a live fraud detector in a modern banking environment.

Because GPT-4 is flexible, SecurePay can check transaction data for any suspicious changes, compared to systems that look only for fixed patterns. As a result, the platform can spot frauds that have not happened before and cut down on false positives by making use of context and language.

SecurePay also increases banking capabilities by bringing together user management, financial analyses and prediction of fraud under the same dashboard for both users and bank administrators. The platform provides people with honest fraud warnings, AI explanations and strong security and it allows administrators to easily manage and observe how the system functions.

4 Ethical Considerations

Since using an AI-powered system like SecurePay raises many ethical issues, those concerns have to be addressed early on to guarantee responsible and clear actions. Taking care of user privacy comes first. It is important for SecurePay to use encryption and authentication methods that cover all their data. Operation Manual should include simple consent policies that meet GDPR and relevant rules, so users understand how their data is being collected, kept and used.

It's also crucial to ensure that algorithms do not discriminate. Bias may be passed from the data used to train the GPT-4 models. It is important for SecurePay to test and check the results of its AI and to teach it using various groups of people. It is important to be sure flagged transactions are handled justly and that users are able to find a solution to any issues.

Being clear and understandable are two main values in ethical AI. SecurePay needs to make sure that the reasons for its fraud detection decisions are clear, enabling both users and administrators to see and understand them. Having a feedback loop will perfect the system and value the choices of the user.

In the end, misuse must be prevented by the system. SecurePay's AI should only focus on identifying fraud and strict security protocols should stop anyone from watching or accessing the AI in an unapproved way. SecurePay can keep its integrity intact and improve financial technology that supports everyone if it follows these ethical rules.

5 Aim

The primary aim of SecurePay is to develop a secure, AI-driven online banking application that detects and mitigates fraudulent financial activities in real-time. Leveraging GPT-4's contextual reasoning capabilities, the system enhances transaction transparency, minimizes fraud risks, and fosters user confidence through intelligent, scalable, and accessible financial monitoring

SecurePay aims to use AI as both a tool that acts behind the scenes and as an assistance tool people use to manage their financial security. It addresses an important issue in the industry by providing explainable, intelligent and fast fraud prevention solutions.

6 Objectives

The objectives behind SecurePay are described to cover both technical performance and security designed for customers:

Set Up Real-Time Detection of Fraud:

You can trust GPT-4 to analyze live activities and notice unusual behaviors, shortening the time for fraud.

Develop an application that provides banking services through all aspects of development.

Use React for your React system, Node.js for your backend and MongoDB/PostgreSQL for securely storing your data on the cloud.

Make it easy for users to interact thoroughly with your app.

Schedule mails with fraud insights for users with AI explanation.

Keep the data safe and comply with regulations.

Based on GDPR and CCPA rules, implement strong handling processes, ensure users consent and include two-factor logins.

Offer analytical systems that are well-equipped for different sizes of organizations regarding fraud.

Design financial analytics tools that display relevant data and make predictions to support your decisions about fraud trends both now and in the future.

Let administrators control user logins and support access from all types of devices.

Build the application to run smoothly on mobile devices, on desktops and tablets and include access controls for users as well as administrators.

7 Risks

Implementing Secure Pay involves several technical, operational, and ethical risks that must be mitigated to ensure project success and user trust:

1. Data Breach Risk High

As the platform handles sensitive financial data, any unauthorized access could compromise user privacy and system integrity. To mitigate this, SecurePay implements end-to-end encryption, secure APIs, regular vulnerability testing, and compliance with data protection regulations like GDPR and CCPA.

2. False Positives and Negatives Medium

AI-based fraud detection may incorrectly flag legitimate transactions (false positives) or miss fraudulent ones (false negatives). Continuous tuning of GPT-4's classification thresholds, user feedback loops, and anomaly review mechanisms will help balance detection accuracy.

3. Model Bias Risk – Medium

GPT-4 may inherit bias from training data, leading to discriminatory risk scoring. SecurePay reduces this risk by using diverse datasets, implementing fairness-aware training procedures, and conducting routine audits of flagged transaction outcomes.

4. System Downtime – Medium

Downtime could delay fraud detection and impact user access. SecurePay employs cloud-based redundancy using AWS, automated backups, and monitoring to maintain high availability.

5. Regulatory Compliance Risk High

Non-compliance with financial and data laws may lead to legal penalties. To prevent this, SecurePay is designed in consultation with legal standards and includes audit logging, user consent flows, and documentation.

6. User Misunderstanding or Lack of Training – Low to Medium

Users may misinterpret alerts or misuse features. Clear UI design, in-app guidance, and support documentation are integrated to educate users and prevent misuse

8 Literature Survey

The evolution of artificial intelligence (AI) has significantly impacted the financial sector, especially in fraud detection and secure transaction systems. As cyber threats become more advanced, financial institutions and fintech companies are increasingly turning to AI-driven solutions to monitor, predict, and prevent fraudulent activity in real time. SecurePay, the proposed platform, integrates OpenAI's GPT-4 model to provide contextual transaction analysis and AI-based anomaly detection, making it a state-of-the-art fraud prevention system. This literature survey explores the core technological and academic concepts supporting the development of SecurePay, including AI in fraud detection, use of language models, financial transaction security, and ethical AI deployment.

8.1 AI in Financial Fraud Detection

According to Bhattacharyya et al. (2011), financial fraud detection typically relies on classification algorithms that identify irregular transaction patterns. Common techniques include decision trees, logistic regression, and neural networks. However, traditional models are often rule-based, which limits their adaptability and accuracy in identifying novel fraud types. In contrast, AI-based systems like random forests and deep learning architectures demonstrate higher precision due to their ability to learn from large datasets and recognize complex, nonlinear relationships between variables.

Recent studies (Bolton & Hand, 2002) have highlighted the growing shift from supervised learning methods to unsupervised and hybrid approaches. Unsupervised models such as k-means and DBSCAN are particularly effective in flagging previously unseen anomalies. These systems do not require labeled data, which is critical when fraudulent examples are rare. SecurePay adopts this hybrid paradigm by integrating a pretrained GPT-4 model with system-specific transaction scoring mechanisms.

8.2 Natural Language Processing (NLP) and Large Language Models (LLMs)

Natural Language Processing plays a vital role in understanding the context of financial communication and transaction descriptions. Transformer-based language models such as BERT (Devlin et al., 2019) and GPT-3/4 (Brown et al., 2020) have revolutionized contextual comprehension in AI systems. GPT-4, the backbone of SecurePay's fraud detection logic, can interpret semantic nuances in transaction metadata, such as transaction type, description, frequency, and user-specific patterns.

Unlike static classifiers, LLMs can reason over context, enabling them to flag suspicious patterns that deviate from a user's norm. This approach is supported by recent findings from OpenAI (2023), which demonstrate that GPT-based models achieve near-human performance in tasks requiring contextual understanding and decision-making. SecurePay applies these capabilities to financial data, using GPT-4 to generate confidence scores and transaction narratives to aid in interpretability.

8.3 Real-Time Transaction Monitoring and Analytics

The capability to monitor transactions in real-time is fundamental to fraud prevention. According to Whitrow et al. (2009), delayed detection mechanisms significantly increase financial risk and losses. SecurePay's real-time fraud analytics system uses stream processing with Node.js and event-driven architecture to monitor transactions as they occur. This aligns with modern architectural patterns in fintech, as documented by Basak et al. (2019), who emphasized the necessity of microservices and event brokers for scalability and resilience.

Moreover, SecurePay incorporates visualization components to make financial and fraud analytics more accessible to end-users and administrators. According to Few (2006), interactive dashboards increase decision-making speed and reduce the cognitive load on security teams. Integrating frontend components built in React with real-time data enables the platform to present alerts, summaries, and charts that aid rapid fraud analysis and user trust.

8.4 Secure Authentication and Data Protection

Security remains a key concern in fintech applications. Studies by Alsmadi & Zarour (2020) emphasize the importance of two-factor authentication (2FA), token-based sessions, and strong encryption standards in reducing system vulnerabilities. SecurePay uses OTPs and TOTP-based 2FA in addition to bcrypt hashing and HTTPS protocols for secure data exchange.

Further, compliance with regulations like the General Data Protection Regulation (GDPR) and California Consumer Privacy Act (CCPA) is critical. According to Shastri et al. (2020), AI systems that handle sensitive data must implement data minimization, transparency, and

explicit consent mechanisms. SecurePay adheres to these practices by requesting clear user consent during registration and maintaining logs for accountability.

8.5 Ethical AI Deployment and Bias Mitigation

As outlined by Binns (2018) and Mehrabi et al. (2021), ethical deployment of AI involves reducing bias, ensuring fairness, and providing explainable results. GPT-based models have shown susceptibility to bias if trained on skewed data. To mitigate this, SecurePay undergoes regular retraining on representative datasets and includes an appeal mechanism for users to dispute flagged transactions.

Explainability is enhanced by GPT-4's ability to produce natural language justifications for its decisions. Research by Ribeiro et al. (2016) through LIME (Local Interpretable Model-agnostic Explanations) has shown that explainability boosts user confidence in AI decisions. SecurePay integrates a similar layer of interpretability, allowing users to see why a transaction was flagged and what contextual cues triggered it.

8.6 ChatGPT-4 as an Intelligent Fraud Agent

While most fraud detection systems rely on numeric models (e.g., credit scoring or pattern similarity), SecurePay innovates by using GPT-4 as an intelligent fraud agent. GPT-4 is capable of understanding user behavior history, semantic labels, and suspicious narratives. A study by OpenAI (2023) demonstrates GPT-4's superiority over earlier models in logical reasoning, semantic consistency, and question answering, which are crucial for fraud interpretation tasks.

SecurePay customizes the prompts and fine-tunes GPT-4 to make risk assessments based on transactional context. This novel integration allows for qualitative fraud detection, where transactions are not just flagged due to numerical anomalies but due to the presence of "unusual language patterns" or deviation from typical contextual behaviour.

8.7 Role of Admin Dashboards and User Interfaces

Admin and user interfaces in fraud detection platforms have been widely studied (Li et al., 2016). These interfaces must be intuitive and data-rich while ensuring privacy. Secure Pay adopts a role-based access control (RBAC) system, enabling administrators to view fraud alerts, system logs, and risk distributions, while limiting sensitive details from general users. This structure follows best practices from ISO/IEC 27001 and NIST cybersecurity frameworks.

Furthermore, dashboards include components for reviewing flagged transactions, user statistics, and AI performance metrics. According to recent UX studies (Nielsen Norman Group, 2021), a well-designed dashboard significantly improves the effectiveness of fraud analysts and lowers cognitive burden.

8.8 Comparative Review of Existing Fraud Detection Platforms

Several commercial fraud detection systems exist, including Kount, Sift, and Feedzai. These platforms provide robust capabilities but are either closed source or highly expensive. Kount uses behavior analysis and device fingerprinting; Sift focuses on trust scores and machine learning models; Feedzai integrates deep learning with user profiling.

However, few platforms use language models for contextual fraud detection. GPT-4's integration in SecurePay distinguishes it by allowing conversational analysis, personalized fraud insights, and proactive learning. Moreover, the open-source architecture of SecurePay provides extensibility and transparency, appealing to small and mid-sized financial institutions

Financial Data Modeling and Transaction Profiling

Modern fraud detection systems rely heavily on modeling user behavior and profiling transaction patterns to identify anomalies. Financial data modeling includes analyzing user-specific transaction frequencies, spending habits, merchant categories, and geolocation trends. According to Jurgovsky et al. (2018), Recurrent Neural Networks (RNNs), particularly Long Short-Term Memory (LSTM) networks, have been effective in capturing temporal dependencies within transaction sequences for fraud detection.

These temporal models allow systems to recognize when a user's transaction deviates significantly from historical norms. For example, an unusually high-value transaction in a new geographic location or merchant type can trigger alerts. SecurePay enhances this by layering contextual NLP-based insights on top of the numerical profiling. The GPT-4 model interprets semantic descriptions of transactions (e.g., "emergency transfer to unfamiliar recipient") to infer behavioral anomalies that may not be numerically evident.

Research by Pozzolo et al. (2017) in highly imbalanced datasets also shows the importance of anomaly detection frameworks like Isolation Forests and Autoencoders, which are useful in fraud cases where genuine transactions vastly outnumber fraudulent ones. SecurePay addresses this challenge by combining GPT-4 for high-level interpretive analysis with anomaly detection techniques for low-level pattern mining.

Additionally, the integration of user-specific risk profiling—such as assigning risk scores based on frequency of disputes or high-volume transfers—is derived from practices used in commercial platforms like Experian and FICO. SecurePay incorporates a similar tiered risk model, adapting dynamically based on user behavior and flagged history.

Role of Explainable AI (XAI) in Financial Decision Systems

As AI systems become more complex, the demand for **Explainable AI (XAI)** in critical applications like fraud detection has grown. Financial institutions must justify why a transaction was blocked or flagged, not just to users but also to regulators. According to Adadi and Berrada (2018), XAI ensures transparency and builds trust in AI-driven platforms by offering human-interpretable explanations for algorithmic decisions.

In fraud detection, explainability is crucial to prevent customer dissatisfaction due to false positives. SecurePay uses GPT-4's ability to generate natural language rationales to provide a unique level of explainability. For instance, instead of presenting a binary flag, the system may say: *"This transaction was flagged because it deviates from your normal behavior and is sent to a new recipient without prior history."*

LIME (Ribeiro et al., 2016) and SHAP (Lundberg & Lee, 2017) are two leading XAI tools frequently used to explain predictions of complex models. While traditionally applied to numerical data, SecurePay’s integration of LLMs enhances interpretability by contextualizing decisions in plain language. This improves customer trust and assists fraud analysts in investigating alerts with greater precision.

Moreover, regulatory standards such as the EU’s GDPR “right to explanation” clause necessitate systems like SecurePay to provide users with clear insights into automated decisions. Studies by Wachter et al. (2017) emphasize the legal and ethical implications of opaque AI models, reinforcing the need for platforms like SecurePay to embed transparency into their core design.

SecurePay’s explainable architecture serves dual purposes: it builds user confidence by demystifying AI decisions and aids developers and compliance teams in auditing the model’s behavior over time

9 Methodology

The development of SecurePay—a GPT-4-powered AI fraud detection banking application—relied on a robust and iterative methodology that blended software engineering best practices, agile principles, API-driven AI integration, and layered system architecture. This section explains the technical development steps, the rationale behind using OpenAI’s GPT-4 API over training custom models, data handling practices, security considerations, system architecture, and challenges encountered during implementation.

9.1 Development Approach

SecurePay was built using a full-stack web development methodology following the Model-View-Controller (MVC) pattern, ensuring modularity, maintainability, and scalability. The core components of the platform include:

- **Frontend:** React.js with Material UI for component styling and responsive layouts.
- **Backend:** Node.js and Express.js to handle REST API routing, server logic, and third-party integrations.
- **Database:** MongoDB (NoSQL) and optionally PostgreSQL (SQL) for structured financial data storage.
- **AI Integration:** OpenAI’s GPT-4 API for real-time fraud detection based on transaction metadata.

- **Deployment:** AWS EC2 (Elastic Compute Cloud) and S3 (Simple Storage Service) for hosting and storage.
- **Security:** JWT for authentication, bcrypt for password hashing, HTTPS for data transmission, and 2FA using TOTP or OTP via SMS/email.

The application followed an Agile Software Development Life Cycle (SDLC), divided into iterative sprints:

1. Requirements Gathering and Planning
2. System Architecture and Design
3. Component-wise Development (UI, Backend API, Fraud AI Integration)
4. Internal Testing and Debugging
5. Deployment and CI/CD setup
6. Feedback and Revisions

9.2 GPT-4 API vs Custom Model: Rationale

A key methodological decision was to use OpenAI's GPT-4 API instead of training a bespoke machine learning model from scratch. This decision was driven by several reasons:

1. Resource Constraints:

Training fraud detection models from scratch requires large volumes of high-quality, labeled transaction data—often sensitive in nature and hard to acquire. In contrast, GPT-4 offers generalized pattern recognition capabilities immediately through secure API calls.

2. Transfer Learning Capability:

GPT-4 is trained on vast datasets and exhibits contextual understanding of data semantics. Its ability to understand transaction patterns, detect inconsistencies, and provide explainable risk scores made it ideal for SecurePay's fraud detection without requiring domain-specific retraining.

3. Explainability and Interpretation:

Unlike black-box ML models, GPT-4 provides coherent, language-based explanations of its predictions. This enhanced the explainability of fraud alerts, making it easier for users and administrators to interpret and act upon flagged transactions.

4. Scalability and Maintenance:

API-based solutions offload model maintenance and retraining to OpenAI, significantly reducing the engineering burden. As GPT-4 evolves, SecurePay benefits automatically from newer improvements without needing system-level code changes.

5. Compliance and Privacy:

Using GPT-4 via an API ensures compliance with global data standards (e.g., SOC 2, ISO 27001), reducing legal risk associated with self-managed ML infrastructure.

9.3 System Architecture

The application was structured as a layered architecture with the following tiers:

1. **Presentation Layer:**
 - React-based frontend
 - Dynamic routing for dashboard views
 - JWT-secured pages based on user roles (Admin/User)
2. **Logic Layer (Middleware):**
 - Express.js handling routing and validation
 - Axios for internal and third-party API communication
 - Middleware enforcing rate-limiting and 2FA requirements
3. **Data Layer:**
 - MongoDB for transaction logs, user credentials, and audit trails
 - Indexed collections for faster lookup during fraud scans
4. **AI Layer:**
 - GPT-4 API integration via HTTPS POST requests
 - Input: anonymized transaction metadata
 - Output: fraud risk score (0–1), explanation string
5. **Security Layer:**
 - Role-based access control (RBAC)
 - Password hashing with bcrypt
 - OTP via Twilio/SendGrid integration
 - HTTPS with SSL

9.4 Data Collection and Processing

To simulate real-world financial behavior for development and testing, a synthetic dataset was generated containing:

- Transaction ID

- Sender/Recipient Name
- Amount
- Transaction Type (e.g., transfer, bill payment)
- Timestamps
- Device used (web/mobile)
- IP address and location
- Previous transaction history

This data was fed into the backend using both seed scripts and live user inputs during internal testing.

Before sending any data to GPT-4, the backend performed **data anonymization** to strip out any Personally Identifiable Information (PII). Metadata such as transaction amount, time, device, frequency, and geolocation was retained for fraud analysis.

Data sent to GPT-4 looked like this:

```
{
  "prompt": "Evaluate the following transaction for fraud: A user in Australia initiated a transfer of $9,500 from a new device at 3:00 AM. Previous transaction average is $200. Location deviation from last known login is 900km.",
  "model": "gpt-4",
  "max_tokens": 100,
  "temperature": 0.2
}
```

GPT-4's response was interpreted to flag the transaction and append its explanation to the transaction history.

9.5 Testing Methodology

Testing was broken down into three phases:

- **Unit Testing:**

For React components, backend endpoints, and MongoDB queries using Jest and Mocha.

- **Integration Testing:**

Ensuring end-to-end data flow, especially transaction creation → GPT-4 evaluation → frontend display.

- **User Acceptance Testing (UAT):**

Multiple role-based test accounts were used to verify user experience, response times, and fraud flagging scenarios.

Edge cases such as unusually high transfers, location spoofing, repeated logins from new devices, and inconsistent transaction timings were used for stress-testing.

9.6 Compatibility and Integration Challenges

Some key challenges included:

1. GPT-4 Token Limits:

The API has input/output token limitations. Careful prompt engineering was required to include only relevant data for fraud evaluation.

2. Latency:

GPT-4 API calls introduced a delay of 1-2 seconds per transaction. To mitigate this, backend asynchronous handling was introduced using a job queue.

3. API Failover:

Fallback mechanisms were developed to continue transactions if the GPT-4 API failed, storing them in a retry queue.

4. Device and OS Compatibility:

Mobile responsiveness issues on Safari and Android required extensive CSS and layout testing.

5. Data Mapping:

Frontend and backend used different field naming conventions. Mapping interfaces were created to translate frontend events to GPT-4-compatible inputs.

Fraud Detection Workflow

The core transaction fraud detection logic is summarized below:

1. Transaction data is validated and sent to GPT-4.
2. GPT-4 returns a contextual evaluation of risk.
3. If risk score exceeds a threshold (e.g., 0.75), the system flags the transaction.

4. Admin receives a fraud alert with justification and options to approve, block, or investigate.

9.7 Feedback Loop and Model Refinement

Even though SecurePay doesn't fine-tune GPT-4 itself, it stores data on flagged transactions and the decisions from administrators to help detect errors. Later releases will see this metadata helping create a scoring system that links GPT-4 reasoning together with adjustments based on rules.

9.8 Future Improvements

- Integration with **federated learning** systems to enable decentralized fraud pattern recognition.
- Use of **LangChain** and **vector embeddings** for semantic similarity checks between transaction histories.
- Inclusion of **voice or biometric login authentication** for high-risk transaction validation.

9.9 Test Plan for the Project

This platform automates fraud detection and also offers all the tools found in online banking. Because it deals with money transactions and personal information, it should be tested to make certain users can trust it and that it is secure when launched. The plan details how to carry out tests, what has to be done, why and when for SecurePay.

Reasons for Testing

The main purpose is to ensure that SecurePay performs as it is expected. This covers creating user accounts, authenticating them, processing transactions, tracking and detecting fraud in the moment with GPT-4's help, delivering notifications and offering administrative tools. Indeed, testing an engine like this also looks at accuracy, so that suspicious transactions are found and approved, minimizing both missed fraud and false alarms for the user. Properly protecting data is very important, achieved by using encryption, ensuring strong authentication and conforming to standards including the GDPR and PCI-DSS. Another significant goal is observing the system's reaction to different workloads, device compatibility and the user experience.

Why do we do testing?

All the key user and administrator features available will be tested in the testing phase. Apart from the activities mentioned above, testers also need to work through signing up, logging in, working with transactions, getting alerts on fraud and handling user or setting administration in the admin panel. The primary aim of integration testing is to test the link between GPT-4 API, fraud detection and All important points, like security, ease of use and performance, will be thoroughly checked. Yet, no crawling for hardware-specific testing or internal testing of third-party services is completed as part of this plan.

Testing Strategy

Testing will be carried out using multiple levels of approach.

- This testing checks if a feature works correctly from entering data to the appearance or reactions of each screen.
- Integration Testing checks whether the API related to fraud detection and notice works with the application's backend services properly.
- During Security Testing, efforts are made to secure the application which covers authentication, authorization, encrypting data and finding vulnerabilities, using OWASP ZAP.
- When you do usability testing, you examine how the user interface works and is displayed on different devices.
- This testing method loads the system with realistic and peak amounts to determine how it deals with stress and measures its reactions as well as its capacity.
- Regression Testing is conducted following updates or bug fixes to see if anything important has changed.

Test Environment

Multiple platforms and devices will be used to conduct testing in real-life situations. Windows, Linux, macOS, iOS and Android are included among Macfuse's supported operating systems. Cross-browser compatibility will be checked by testing Chrome, Firefox, Safari and Edge browsers. Services are provided by Node.js in the backend, together with MongoDB and PostgreSQL as databases. To protect the production data, sandbox or test keys will be applied when using the GPT-4 API.

Test Cases

For example, verification involves making sure user registration works, users are confirmed by email, login works correctly and transactions are properly handled. Certain tests will make sure that the AI sends alerts when it finds something suspicious. Unauthorized attempts will be made to access admin routes and password resets will be checked during security tests. Simultaneous user access during load tests will determine the system's stability. The expected results and the actual results will be entered into the test documentation for every case.

What Managers Do

The Test Manager will take charge of managing and organizing the tests. To test an application, Test Engineers prepare test scenarios, assess problems and confirm that these fixes are functioning as intended. Problems found should be fixed by the developers, who will also support the testers when required. Penetration testing and vulnerability analysis will be the main tasks for Security Analysts. Evaluating usability and accessibility can be done more effectively with input from UX designers.

Schedule

The test process divides into phases, starting with planning and setting up the environment in the first week and then spending the following two weeks on making detailed test cases. Functional, integration and security tests are scheduled for three weeks and performance and usability tests will take an additional week. Regression tests will take place continually during the cycles of development. In the final week, reports will be prepared, and stakeholders will be asked to sign them.

10 Requirements

To create a reliable, secure and helpful platform for users, the SecurePay system meets all applicable functional and non-functional requirements close to AI-based fraud detection. The system is required to ensure users are registered and authenticated safely and that their access to the system is prevented by using multi-factor authentication. There are users and administrators in this system. Users take part in financial actions, view information and get instant fraud alerts, while administrators look after users, set up fraud policies and track suspicious activity. Every single transaction is checked right away using AI against any signs of fraud. It also includes mechanisms to tell both users and admins when there might be fraud or important events. Non-functional requirements involve setting up security features, following regulations such as GDPR and PCI-DSS and assuring performance by making the system fast and constantly accessible. When there are more users, the system can grow by splitting the database and scaling sideways. You improve maintainability by having clean, modular code and good documentation and usability increases when the UI is responsive and considers all kinds of users. Part of data management is protecting transaction records, audit log information and data from AI systems, with detailed guidelines on what data should be both kept and deleted.

Category	Requirement	Description
Functional	User Registration & Authentication	Secure signup/login with multi-factor authentication and password hashing.
	User Roles & Access Control	User and Admin roles with different permissions and access levels.
	Transaction Management	Initiate, validate, and store transactions with metadata like timestamp, device, location.
	Real-Time Fraud Detection	Analyze transactions using GPT-4 API, flag suspicious transactions, and alert users/admins.
	User Dashboard & Analytics	Display transaction history, fraud alerts, financial insights with visual charts.
	Admin Management Features	User management, fraud alert management, system configuration, and reporting.

Non-Functional	Security	End-to-end encryption, JWT authentication, role-based access control, and regulatory compliance (GDPR, CCPA, PCI-DSS).
	Performance	Low latency in transaction processing and UI responsiveness.
	Scalability	Support horizontal scaling, database sharding, and caching for increased load.
	Availability & Reliability	Target 99.9% uptime, backup strategies, and monitoring.
	Maintainability & Extensibility	Modular codebase with clear documentation and support for new features.
	Usability	Responsive, accessible UI compliant with WCAG 2.1 standards.
Data	Transaction Records	Store detailed transactions, metadata, fraud scores, and statuses.
	User Profiles	Personal info, credentials, roles, and security settings.
	Audit Logs	Immutable logs of admin/system actions and flagged transactions.
	AI Interaction Logs	Data for GPT-4 API inputs/outputs and risk scoring.
Compliance	Regulatory Adherence	GDPR, CCPA, PCI-DSS compliance with data protection and privacy.
	Data Retention Policies	Secure deletion and user consent management.

11 Design

The Secure Pay platform design is structured around a modular and scalable system architecture that integrates AI-driven fraud detection seamlessly into banking operations. The frontend is developed using React with Material UI, ensuring a responsive and accessible user interface that provides dashboards, transaction management, and administrative controls with intuitive navigation. The backend uses Node.js and Express to handle API requests, business

logic, and secure data transactions, connecting to MongoDB and PostgreSQL databases for storing user profiles, transaction records, and audit logs. A critical component of the design is the integration of the GPT-4 API, which analyzes transaction data in real time to flag suspicious activities, offering an adaptive and intelligent fraud detection layer. Security is embedded across the system with JWT-based authentication, password hashing, encrypted data storage, and role-based access control, complemented by optional two-factor authentication for enhanced protection. The design WebSocket support to facilitate real-time updates and alerts. Overall, the system is engineered for high availability, compliance with data protection laws, and scalability, while emphasizing a clean user experience that empowers users and administrators to interact with the platform effectively

Aspect	Description
System Architecture	- Frontend: React + Material UI for responsive UI. - Backend: Node.js + Express for API and business logic. - Database: MongoDB & PostgreSQL. - AI Layer: GPT-4 API for fraud detection. - Notification Module: Email, SMS, and in-app alerts. - Security Module: Authentication, encryption, 2FA.
Frontend Components	- Login/Register pages with validation. - Dashboard with overview, transaction history, analytics. - Transaction creation form. - Admin panel for user/alert management.
Backend Services	- Authentication and authorization. - Transaction processing and validation. - Fraud detection API integration. - Notification dispatch. - Admin controls and reporting.
Database Design	- Users Collection: user info, roles, security data. - Transactions Collection: transaction metadata, fraud scores. - Fraud Alerts: flagged transaction logs. - Audit Logs: admin and system events.
Security Design	- HTTPS enforced. - JWT-based authentication. - Passwords hashed with bcrypt. - Data encrypted at rest. - Role-based access control. - Optional 2FA.
UI/UX Design	- Responsive layouts using Material UI grid. - Accessibility compliant.

	- Clear fraud alert visualization. - Sidebar navigation with active state highlighting.
Integration Design	- Secure GPT-4 API calls with rate limiting. - Third-party services for SMS/email notifications. - Planned WebSocket support for real-time updates.

12 Glossary

AI (Artificial Intelligence)

Simulation of human intelligence in machines programmed to learn and solve problems.

API (Application Programming Interface)

A set of protocols allowing different software components to communicate.

Authentication

Process of verifying the identity of a user or device.

Authorization

Process of granting or denying access rights to resources based on identity.

Bcrypt

A password hashing algorithm designed to be computationally expensive to deter brute-force attacks.

CCPA (California Consumer Privacy Act)

Privacy law enhancing data privacy rights for California residents.

Dashboard

User interface that organizes and presents key information and metrics.

False Positive

A result where a benign transaction is incorrectly flagged as fraudulent.

Fraud Detection

Techniques to identify potentially unauthorized or malicious transactions.

GDPR (General Data Protection Regulation)

European Union regulation for data protection and privacy.

JWT (JSON Web Token)

A compact token format for securely transmitting information between parties.

MFA (Multi-Factor Authentication)

Security mechanism requiring multiple verification methods.

MongoDB

A NoSQL document-oriented database program.

Node.js

JavaScript runtime environment used for backend development.

OpenAI GPT-4

A large language model API by OpenAI used for advanced natural language processing.

PCI-DSS (Payment Card Industry Data Security Standard)

A set of security standards to protect card payment data.

PostgreSQL

An open-source relational database system.

REST API

An architectural style for designing networked applications.

Role-Based Access Control (RBAC)

Authorization method restricting system access to authorized users based on roles.

TOTP (Time-based One-Time Password)

A temporary password generated using the current time and a shared secret.

Two-Factor Authentication (2FA)

An additional layer of security using two different authentication factors.

UX (User Experience)

The overall experience a user has when interacting with a system or product.

Wallet

A digital tool or platform used for managing funds and transactions.

13 References

- Bhattacharyya, S. *et al.* (2010) 'Data mining for credit card fraud: A comparative study,' *Decision Support Systems*, 50(3), pp. 602–613. <https://doi.org/10.1016/j.dss.2010.08.008>.
- Binns, R. (2017) *Fairness in Machine Learning: Lessons from Political Philosophy*. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3086546.
- Bolton, R.J. and Hand, D.J. (2002) 'Statistical Fraud Detection: A review,' *Statistical Science*, 17(3). <https://doi.org/10.1214/ss/1042727940>.
- Brown, T.B. *et al.* (2020) 'Language Models are Few-Shot Learners,' *arXiv (Cornell University)* [Preprint]. <https://doi.org/10.48550/arxiv.2005.14165>.
- Devlin, J. *et al.* (2018) 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,' *arXiv (Cornell University)* [Preprint]. <https://doi.org/10.48550/arxiv.1810.04805>.
- Mehrabi, N. *et al.* (2021) 'A survey on Bias and Fairness in Machine Learning,' *ACM Computing Surveys*, 54(6), pp. 1–35. <https://doi.org/10.1145/3457607>.
- Ribeiro, M.T., Singh, S. and Guestrin, C. (2016) '“Why Should I Trust You?,”' *Non Name*, pp. 1135–1144. <https://doi.org/10.1145/2939672.2939778>.

Appendix A – dashboard overview(admin)

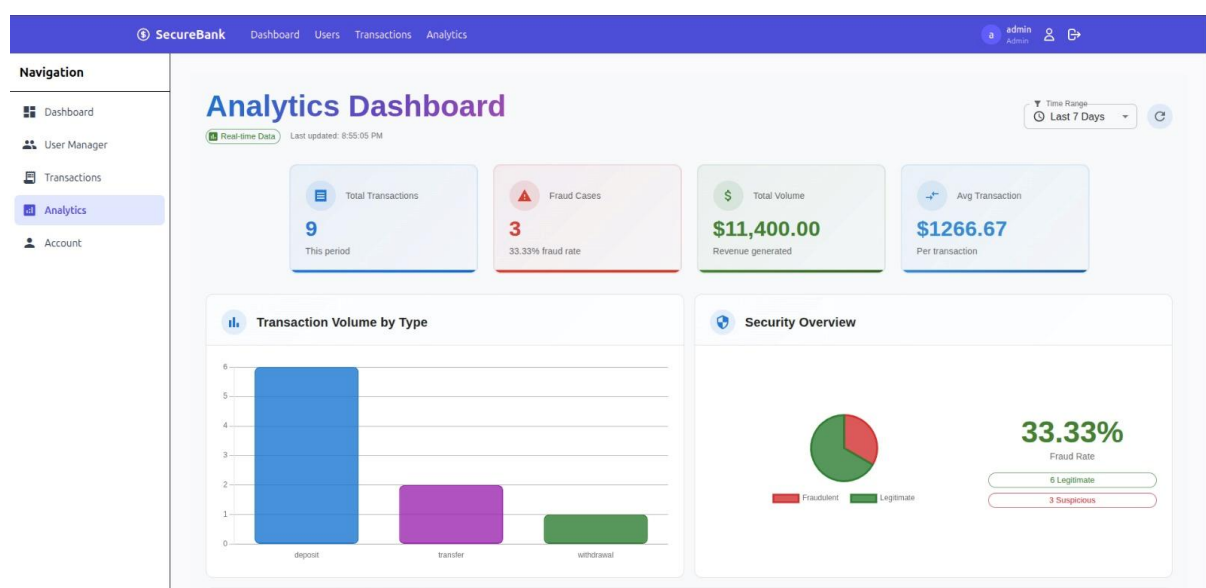


Figure 1 admin analytics overview

Appendix – B

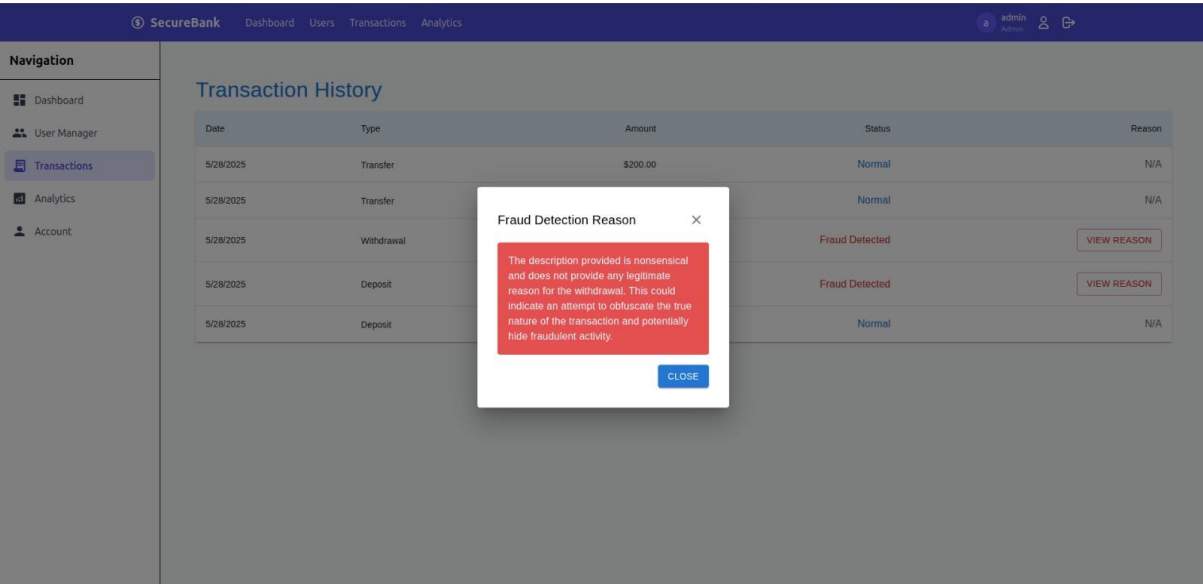


Figure 2 fraud detection